

Solutions Chalmers Challenge 2026

1 Competitive Morality

Competitive Morality

Author: Joshua Andersson

Description

You have a red and a blue button. Every person in the world except for you have pressed either red or blue before you, and you know how many pressed each one. If at least half people press blue, all blue pressers survive, otherwise they die. Which one should you press to maximize the number of people who survive in total?

Subtask 1/2: all red or all blue

First, realize that you are a person. Thus, it is sometimes best to just save yourself.

Thus, in these two subtasks, always pressing red works:

- If everyone presses red, you should press red and save yourself
- If everyone presses blue, the blue will all survive. Any button works

```
print('RED')
```

Full solution

Most of the time, pressing red is better. However, sometimes we have the option to tip the scales, and us pressing blue makes them survive. For example: 4 red, 3 blue. If we press blue, everyone lives. If we press red, 3 die.

Thus, a working strategy is to press blue if, after us pressing blue, the blue will survive. Otherwise, press red.

```
r=int(input())
b=int(input())

if b+1 >= r:
    print("BLUE")
else:
    print("RED")
```

2 Three flavors

Three flavors

Author: Joshua Andersson, Julia Mårtensson

Description

Given a string consisting of characters a, b, c, determine whether they can be ordered such that all adjacent characters are different.

Example input string:

aab

A valid ordering is

aba

Subtask 1: only **a** and **b**

If the number of a is equal to number of b , we can alternate

ababababab

By the pigeonhole principle, a solution only exists if $|a - b| \leq 1$. The pattern above is not enough, though: if there are more b 's, the first characters must instead be a b :

babab

If we started with **a**, we would get stuck:

ababb

Subtask 2: naive solution, $|S| \leq 10$

- Create all permutations of the string
- For each permutation: check that no neighbours have the same color
- Return the first valid permutation, otherwise IMPOSSIBLE

Full solution

WLOG, assume $a \leq b \leq c$ (counted in number of occurrences). In some sense, c is the hardest to place without having two adjacent c 's. Let's try to fit as many c as possible

c.c.c.c.c.c.c.c.c

This packing “clearly” maximizes the number of c . Thus, if it doesn't fit, it's impossible. Otherwise, it's possible.

First place c

c.c.c.c....

Then place b backwards (start at the rightmost free spot, then move either 2 or 3 steps left)

c.cbcbcb.b.b

and finally, we can just place the a 's in the remaining slots. These cannot collide.

cacbcabcabab

3 Stirring Porridge

Stirring Porridge

Author: Julia Mårtensson

Description

Given T seconds, find the largest k such that $\left(\sum_{i=1}^k i = \frac{k(k+1)}{2}\right) \leq T$.

Solution

Since $T \leq 10^6$, a simple while loop adding $1 + 2 + \dots + k$ until the sum exceeds T is enough.

Can also be solved in $O(\log(T))$ using binary search.

Even faster: solving $\frac{k(k+1)}{2} \leq T$ directly gives $k = \left\lfloor \frac{-1 + \sqrt{1+8T}}{2} \right\rfloor$ in $O(1)$. For large T , precision might be problematic. Any arbitrary-precision library with sufficient precision guarantees should work well.

4 Competition Rivalry

Competition Rivalry

Author: Ryan Tan, Rasmus Hulthe

Description

Given Ellinor and Ryan's submissions to a contest, simulate whether Ellinor strictly wins for each of the 3 given tiebreaker rules: ICPC penalty, submission count and time of last score increase

Subtasks

Subtask 1: $n, x, y \leq 100$

Any correct solution works. We can iterate through each problem, filter all submissions for that problem, and compute the required statistics directly.

Subtask 2: Ellinor and Ryan submits at most one submission to a problem

This means that for each problem, the one submission that Ellinor or Ryan makes would provide the best score for the problem. There are no prior submissions to be penalised, the submission count = number of problems attempted, and the latest best-score time is just the latest submission time.

Subtask 3: Total scores of Ellinor and Ryan are different

In this case, we do not need to apply the tiebreaker rules, just iterate through the submissions to get the maximum score of Ellinor and Ryan for each problem, and sum them up respectively and compare.

Full solution

For each contestant, sort submissions per problem by ascending time. Let t be the total prior submissions before the current best score's first occurrence, and c be the submissions since the last score improvement.

Do a linear pass to get t and c for the whole problem itself. While iterating, if a solution beats the current best score, increment t by c as all submissions since the last improvement are now “prior” to the new best, and c resets to 0.

From this, all earlier submissions before the best score end up in t . Submissions after the final best score stay in c and are never added.

Given t and the best score for all problems each contestant did, we can calculate and compare rule 1. To compare rule 2, just take the total amount of submissions. To compare rule 3, we just store the earliest time where the best score was achieved using the linear pass previously.

5 Skimaxxing

Skimaxxing

Author: Ryan Tan, Rasmus Hulthe

Description

You are given m mountains ranked by difficulty (then altitude) and a favorite mountain f . For each query you are given x . Filter out mountains with altitude $< x$; determine if f is in the top k of the remaining available mountains. There are 5 difficulty levels.

Subtask 1: $m, y \leq 1000$

Sort all mountains by primary key descending difficulty, and secondary key descending altitude. This gives us the distinct ordering that we want.

For each query x_j , loop through all mountains starting from the first mountain in the ordering. Keep a counter c of how many mountains have been selected. If a mountain is taller than x_j , increment c by one. Stop when we reach the favourite mountain or when $c = k$, whichever comes first.

If $c = k$ and we do not see the favourite mountain while iterating through the mountains yet, that means that the k mountains have already been selected, and Abdullah's favourite mountain is not included. Thus, the answer for the query is "NO".

If $c < k$ but the favourite mountain's altitude $< x_j$, then that means Abdullah is unable to ski on his favourite mountain. Thus, the answer for the query is "NO" as well.

Otherwise, Abdullah has not found k mountains yet, and can take his favourite mountain as part of it. As such, the answer for the query would be "YES".

The time complexity of this solution is $O(my)$.

Full solution

Binary Search

Instead of brute forcing through the entire mountain list, if we split the list into 5 sublists based on the difficulty level and sort them by descending altitude, they are monotonic, as all mountains before a point in the sublist has altitude $\geq x_j$, and all mountains after has altitude $< x_j$.

We can use binary search on each sublist to figure out how many mountains of a higher difficulty has a high enough altitude to be added before we get to the favourite mountain. This helps us increment c in $O(\log m)$ instead of $O(m)$.

The overall time complexity is $O(m \log(m) + 5y \log(m))$.

Alternate solution

Heap

Alternatively, we can get a list of the queries, and sort them by ascending x_j . We can still use the complete mountain list with ordering of all mountains by difficulty then altitude from the earlier subtask.

For the first year in the queries list, we iterate through the mountains starting from the highest difficulty sublist, and add mountains in a minimum heap with altitude key until the heap has k mountains. These are the k mountains which will be skied on for the year.

Recognise that all skipped mountains have an altitude $< x_j$, which only increases in subsequent queries.

By using a minimum heap, when we go to the next year, simply extract all mountains from the heap that have an altitude $< x_j$, and iterate through the mountains that have not been checked yet, until we get k mountains. We will never add removed mountains back for future queries.

If the favourite mountain is in the heap for a query, the answer for that query is “YES”, else “NO”.

The time complexity is $O(m \log(m) + y \log(y) + m \log(k))$.

6 Counter Attack

Counter Attack

Author: Rasmus Hulthe, Joshua Andersson

Description

Given the Power of your and the enemies creatures in MTG. Figure out if you can survive AND win on the next turn.

Subtask 2: $M, N \leq 2000$

A lot of the choices we can make can be resolved greedily. Suppose we block with K creatures. Then, we ought to take our K weakest creatures and use them to block Rasmus' K strongest creatures.

However, it's not completely obvious what K should be. One idea is to make K the smallest possible such that we survive (after all, us having extra health is useless). Unfortunately, lifelink makes this strategy not work, as Rasmus might end up healing a lot. One full solution is to keep losing health as long as Joshua gains more attack from surviving creatures than Rasmus gets healed via lifelink.

Let's instead loop over how many creatures to block with, checking whether we win in each one. It's also useful to first sort Rasmus' creature strengths in descending order and Joshua's in ascending order. This runs in $O(N(N + M) + M \log(M))$.

Subtask 3: All non-Wall creatures have Power 1

This subtask is intended to be easier to implement. You don't need to track sums of powers, only amounts of creatures.

In this subtask, the strategy to block the smallest amount to keep yourself alive works.

Full solution

We will once again brute over the number of creatures to block with. However, we will aim to do it in $O(N \log(N))$ time total instead.

Loop over every possible value of K , maintaining the sum of the top K strongest Rasmus creatures and K weakest Joshua creatures. If we also precompute the sum of the attack of Joshua's and Rasmus' creatures, along the sum of strength of Joshua's K strongest creatures, we have enough information to check each K in $O(1)$.

$$O(N \log(N) + M \log(M))$$

7 Inverse LIS

Inverse LIS

Author: Joshua Andersson

Description

Given the LIS (longest increasing subsequence) DP table of a permutation, restore the permutation (or one with the same LIS DP table).

Subtask 2: $L(\pi)_i \leq 2$

What does each $L(\pi)_i = 1$ mean? That number i is smaller than all previous numbers. If this was not the case, $L(\pi)_i$ would be equal to 2.

What does each $L(\pi)_i = 2$ mean? That there exists a number j where $j < i$ and $\pi_j < \pi_i$. In words, there's some number to the left that's smaller. Critically, there must not be a smaller number to the left with $L(\pi)_j = 2$, since that would imply $L(\pi)_i = 3$.

To keep the 2s happy, we can assign N to the first 2, $N - 1$ to the second 2, etc. This way, no 3 can ever be formed.

To keep the 1s happy, we can assign the largest unused numbers to the first 1, the second largest to the second 1, etc. This way, the ones will actually get value 1, while all twos will get value 2.

This satisfies all constraints.

Full solution

Just generalize the previous construction. Assign N to the largest number, $N - 1$ to the second occurrence of the largest number, etc.

An easy to code solution that does this is to sort the input by $(L(\pi)_i, -i)$ in increasing order, and assign 1 to the first i in this order, 2 to the second, etc.

```
n=int(input())
lis_pi = [(x, -i) for i, x in enumerate(map(int,input().split()))]
lis_pi.sort()

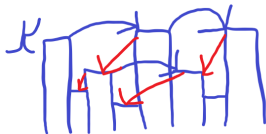
ans = [0] * n
for i, (_, orig_i) in enumerate(lis_pi):
    ans[-orig_i] = i+1
print(*ans)
```

Alternative solution

Let's try to solve the problem by doing topological sorting on a graph. Create a node for each index. An edge $u \rightarrow v$ means that u must have larger value than v . First, for every k , take all i where $L(\pi)_i = k$ and draw edges between adjacent ones. This ensures that none of them get a larger value than they should.



Then, to ensure that each one with value $L(\pi) = k$ actually gets this value, we can draw an edge to some index with value $K - 1$. We can add the closest one to the left. We can then topologically sort this graph to find a solution.



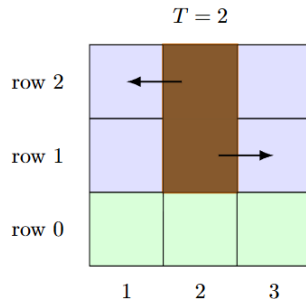
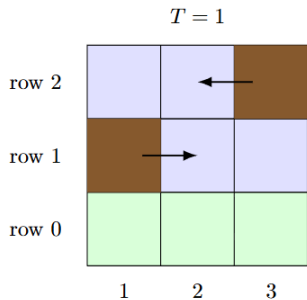
8 Crossy River

Crossy River

Author: Joshua Andersson

Description

There is a $R \times C$ ($R = 1000, C \leq 10^9$) grid. At some moments in time, logs appear at the start or end of some row, flowing at a constant velocity to the other side. Is there an integer point in time where there is a column, where every cell in the column has a log in it?



Subtask 1: $N \leq 500$

The answer is **NO**: we need at least 1000 logs.

Subtask 2: $N, C, t \leq 2000$

Simplification

Instead of coding it as logs appearing at certain times, start them outside of the grid, with an appropriately adjusted column such that they appear at column 1 or C at the exact right time. For each log, we now only need its row, column and velocity (positive or negative).

Subtask

The grid becomes empty within 2000 timesteps. For each timestep, check in $O(N \log(N))$ whether there exists a filled column using e.g. sets.

Subtask 3: $N \leq 2000$

Let's randomly choose two logs i and j . Because $R = 1000$, there is a $\frac{1000}{N^2}$ probability of choosing a pair from the time when there exists a bridge. Let's intersect i and j : they either meet at exactly one point in time, never, or infinitely often.

- If they never meet, continue
- If infinitely often, they are always on the same column. Check if there exists enough blocks that are always aligned with these two to win. If not, continue.
- Otherwise, the solution has at least two non-aligned blocks that meet at exactly one point in time. By intersecting i and j , we now have a candidate column and time. We can in linear(ithmic) time check if there exists a bridge.

In expectation, we will need to sample $O\left(\frac{N^2}{1000}\right)$ pairs, each taking $O(N \log(N))$ to check, for

$$O\left(\frac{N^3 \log(N)}{1000}\right)$$

total. The log factor can be avoided by using hash tables.

Full solution

($N \leq 20,000$ is for full solutions that are poorly implemented)

There exists at least one row that has at most $\frac{N}{1000}$ logs. For each log in this row with least logs: once again, first check if “parallel” logs are a solution. Otherwise, intersect it with each other log: group intersections by time and check if some time works.

$$O\left(\frac{N^2 \log(N)}{1000}\right)$$

The log factor is from set/sorting. It can easily be saved, since you can also use hashtables instead (be mindful of the fact that C++ `unordered_set` is very slow). The time limit is a little tight, so be mindful of the constant factor of your solution. One good option is to use radix sort.

You can also do a sweep, sweeping over row 1, 2, ..., maintaining partial bridges.

Smaller R ?

This problem is almost certainly impossible if R can be small: $R = 3$ is the hardest possible. It is believed that this case cannot be solved in subquadratic time.

To see why, we can loosely rephrase the problem as: given a bunch of colored lines, is there a coordinate (x, y) , where exactly R different colors intersect this point? From here, it should not be hard to do a reduction to the problem “given N lines, is there a triple that intersect at a point?”, which is 3SUM-hard.

We also believe that $R = 2$ is impossible: we are not aware of how to find any pair with an integer intersection point.

9 Mighty Strings

Mighty Strings

Author: Joshua Andersson

Description

You are given a collection of strings U . A string is mighty if each of its substrings appears in U at least once. Determine the number of mighty strings in U .

Subtask 1-3: $T \leq 10^5$

For each string, check if it is mighty. Start by checking all substrings of length 1, then length 2, etc. Break early if some substring is missing. To check whether a substring is in U , we can use string hashing:

<https://cp-algorithms.com/string/string-hashing.html>.

Surprisingly, this runs in $O(T\sqrt{T})$. The worst case looks something like a , aa , aaa , $aaaa$, ..., which takes $O(T\sqrt{T})$ time.

The total length of $a^1, a^2, a^3, \dots, a^k$ is about k^2 . The largest possible k is the solution to $k^2 = T$, which is $\sqrt{T}$. We thus have about $\frac{T}{\sqrt{T}} = \sqrt{T}$ strings, each taking about $\sqrt{T}^2$ time to check all substrings, for a total of $O(T\sqrt{T})$.

Full solution: $T \leq 10^6$

Insight: if a string is mighty, then all of its substrings must also be mighty.

This allows us to do a DP: to check whether S is mighty, check whether each substring of length $|S| - 1$ is also mighty. This works, since we implicitly check every other substring of S by checking those with length $|S| - 1$. Since there are exactly two of these, it is enough to extract the two strings, and check whether they are mighty. To store the DP table, you can either maintain a set of strings for $O(T \log(T))$, or a hashset of strings using string hashing for $O(T)$ total.

Example:

```
abcdef  
  bcdef (first with length |S|-1)  
abcde  (second with length |S|-1)
```

If you do it bottom-up, a valid DP order is to process them grouped by length.

10 Mightier Strings

Mightier Strings

Author: Joshua Andersson

Description

You are given a bunch of strings U . A string S is mighty if each of its substrings appears in U at least as many times as it appears in S . Determine the number of mighty strings in U .

Subtask 1 – 3

We can find two useful bounds. For a string S , if

- $\frac{|S|(|S|+1)}{2} > |U|$, or
- $\frac{|S|(|S|+1)(|S|+2)}{6} > T$

then S cannot possibly be mighty. The first one comes from it requiring too many substrings. The second one comes from it requiring substrings of too long total length. Note that these bounds don't apply to the previous problem.

By filtering using these, we only need to check strings of length up to $\sqrt[3]{T} \approx 83$. The worst case is thus (approximately) achieved by the input consisting of as many strings of length 83 as possible, about $\left(\frac{10^5}{83}\right) * 83^2 \approx 10^7$ substring checks.

Thus, simply checking all substrings with string hashing is sufficient for subtasks 1 – 3.

Subtask 4

To solve subtask 4, it suffices to either implement a highly optimized version of the solution for subtask 3, or write an $O(T \log(T))$ solution. Unfortunately, despite $O(T \log(T))$ solutions being “obviously” better, it is difficult to distinguish between the two, as they run approximately as fast.

Full solution

To solve $T = 5 * 10^6$, we need to use DP ideas to run in $O(\Sigma * T)$ (where $\Sigma = 26$) instead of $O(T \sqrt[3]{T})$.

Instead of combining $S[1 :]$ and $S[: -1]$ as we did in the previous problem, let's try to compute enough info in $S[1 :]$ to check if S is mighty quickly. Adding a new character to $S[1 :]$ creates $|S|$ new substrings, so if we can reuse information well enough without blowing up our memory usage, this should work.

A memory-optimized DP works as follows. Let's build a trie of all strings in U . In each node, store how many times the corresponding string appears, `cnt`. Let's do a single DFS to compute for each node in the trie whether it's mighty.

Some properties:

- If a node in the trie has `cnt=0`, we can instantly return: the 0 means that this string is missing. Thus, any string that has this string as a substring cannot be mighty.
- We can afford to do a linear amount of work per node! (Linear as in the depth of the node). Why? Because if we are at depth D , to get this node, we must have “paid” D characters to get this deep by having this string in the input.
- Even further, for a string of length D , we get $\binom{D+1}{2}$ operations across all of its substrings.

Full solution (ii)

Let's do a DFS, where we break as soon as the current node is not mighty. If we are at a certain node and want to add a new character, we only need to check suffixes of the current string plus the new character.

Suppose we are at node u . To be able to efficiently check for each child whether it is mighty, we maintain two things:

- For each substring of u , how many, times has it been used? We store this information in the corresponding nodes in the trie.
- For each suffix of u , which node does it correspond to in the trie?

Suppose we add a new character c . First, we check that every suffix can be extended with c . If some substring does not exist enough times in the input, break. Otherwise, advance all of the current suffixes by c , update node counts and add a new suffix, the node c (from the root of the trie).

From our earlier bounds, the list of suffixes stays small, so that part is cache-friendly. We do not need to create copies of it: when we go back up in the trie, we can just pop the last element, and let every node go up to their parent, and decrement node counts. It is also possible to optimize the solution to $O(T \log(\Sigma))$.

11 Solving Problems

Solving Problems

Author: Rasmus Hulthe, Ryan Tan

Description

Given N problems forming a prerequisite tree, each with a score S_i and a risk $A_i\%$ of getting stuck (ending the competition), find the ordering that maximizes expected score.

Core Idea: The Exchange Argument

Since every problem has positive score and positive success probability, it is always beneficial to include all N problems in our sequence (we can always append a new problem at the end for positive marginal gain). The only question is: **in what order should we attempt them?**

Define for a **block** B (a contiguous subsequence of problems attempted in some fixed order):

- $E(B)$: the expected score from attempting the block
- $Q(B)$: the probability of surviving (passing) all problems in the block

When we concatenate block A then block B :

$$E(A + B) = E(A) + Q(A) \cdot E(B), \quad Q(A + B) = Q(A) \cdot Q(B)$$

When should A come before B ? Comparing $E(A) + Q(A) \cdot E(B)$ vs $E(B) + Q(B) \cdot E(A)$, we find A before B is better iff:

$$\frac{E(A)}{1 - Q(A)} > \frac{E(B)}{1 - Q(B)}$$

Core Idea: The Exchange Argument (ii)

We call $\text{eff}(B) = \frac{E(B)}{1-Q(B)}$ the **efficiency** of a block. Higher efficiency should come first.

Subtask 1 (Single chain, 5 pts)

With $P_i = i - 1$, the problems form a single chain $1 \rightarrow 2 \rightarrow \dots \rightarrow N$. The only valid ordering is the chain itself, and including all problems is optimal (each has positive marginal expected value). Simply compute:

$$E = \sum_{j=1}^N S_j \cdot \prod_{m=1}^j q_m$$

where $q_i = 1 - \frac{A_i}{100}$ is the success probability.

Subtask 3 (No prerequisites, 20 pts)

With $P_i = 0$ for all i , problems are independent. By the exchange argument, we sort by decreasing efficiency $\text{eff}(i) = \frac{S_i \cdot q_i}{1 - q_i}$ and attempt them in that order.

$$E = \sum_{j=1}^N S_{\pi_j} \cdot \prod_{m=1}^j q_{\pi_m}$$

where π is the sorted permutation. This runs in $O(N \log N)$.

Subtask 5 ($N \leq 10$, 5 pts)

With at most 10 problems, we can try all $O(N!)$ orders to solve them in (and making sure to discard orders that don't respect prerequisites). It is also possible to solve this subtask in $O(2^N * N)$.

Full solution:

To achieve the optimal ordering under tree constraints, we extend the exchange argument using a greedy merging strategy.

If a problem v has the highest efficiency among all available problems, it is strictly optimal to attempt v immediately after its prerequisite u . Therefore, we can treat u and v as a single contiguous block of problems.

Algorithm details:

1. Initialize each problem as its own block with expected score $E(i) = S_i \cdot q_i$ and survival probability $Q(i) = q_i$.
2. Maintain a max-heap of all valid blocks, ordered by their efficiency $\frac{E(v)}{1-Q(v)}$.
3. Extract the block v with the maximum efficiency.
4. If block v has already been merged into another block, discard it and continue.
5. Find the current block u that contains the prerequisite of v . A union find with path compression efficiently tracks these merged blocks.
6. Merge v into u . The new combined block u' updates the expected score and survival probability:

Full solution: (ii)

$$E(u') = E(u) + Q(u) \cdot E(v)$$

$$Q(u') = Q(u) \cdot Q(v)$$

7. Update the union find so that v 's parent points to u .
8. Insert the newly formed block u' back into the priority queue (unless u is the dummy root node 0).
9. Repeat until all problems are merged into the root. The final answer is $E(0)$.

Why does this work? If a certain problem v has the best efficiency currently, then it is optimal to take it instantly when it becomes available (we do not assume anything about when it should become available). Thus, this means that after we take its parent, we instantly take v . If we remove v from the tree, and update the parent node's value to include taking v , we get a new, slightly smaller tree, with the same answer as the original tree. We can then repeat this procedure until the tree consists of a single node.

Since we have a forest of rooted trees, a convenient trick is to add a single fake problem with $S = 0$ and $q = 1$. Doing this, we don't have to sort all the roots of all trees we end up with at the end. Just be careful to not divide by 0, since $q = 1$.

12 The Cursed Archipelago

The Cursed Archipelago

Author: Rasmus Hulthe

Description

A Pirate has hidden coins on up to $Z \leq 100$ out of $N \leq 10^5$ islands. You have $Q \leq 400$ queries to pin down which islands have coins, and exactly how many. The Pirate can lie up to $L \leq 100$ times. A query is made up of N multipliers, one for each island. The Pirate will multiply the amount of coins on island i with multiplier X_i , sum these up and tell you this sum modulo $10^9 + 7$, i.e. the dot product (mod p).

Subtask 1: $Z \leq 10, L = 0, Q = 300$

Here the Pirate never lies. This is very nice.

We can do a divide and conquer or binary search.

By creating a query X where $X_i = 1$ for a range of islands and 0 elsewhere, the result will be non zero if and only if there is gold in that range.

We can isolate each island with gold in roughly $\log_2(10^5) \approx 17$ queries per island.

This takes at most $10 \times 17 = 170$ queries, so 300 is very lenient.

Subtask 2: $Z \leq 100$, $L = 0$, $Q = 200$

Still no lying, but now up to 100 islands and only 200 queries.

Binary search is now too hungry, $100 \times 17 > 200$.

However, because the Pirate still doesn't lie we can do some algebra to solve it exactly.

Let the islands with gold be located at indices $p_1, p_2, \dots, p_Z$, and let the amount of gold on them be $a_1, a_2, \dots, a_Z$.

If we set our query multipliers to $X_i = i^q$, we are asking the Pirate to compute:

$$S_q = \sum_{i=1}^Z a_i (p_i)^q$$

For $q = 0, 1, 2, \dots$, this sequence $S_0, S_1, S_2, \dots$ is known as the **moments** of the sequence

Why do we want moments? Because sequences formed by summing powers always satisfy a linear recurrence relation!

Subtask 2: $Z \leq 100, L = 0, Q = 200$

This means there exist some hidden coefficients $C_1, C_2, \dots, C_Z$ such that:

$$S_q = C_1 S_{q-1} + C_2 S_{q-2} + \dots + C_Z S_{q-Z}$$

This is something that can be solved! The Berlekamp massey algorithm takes $2Z$ moments, (200 queries).

And it finds the shortest linear recurrence that generates our sequence (and if given $2Z$ moments, it is unique).

Even more, it finds the characteristic polynomial:

$$P(x) = x^Z - C_1 x^{Z-1} - C_2 x^{Z-2} - \dots - C_Z$$

The roots of this polynomial will be the exact locations of the islands with gold on them, $p_1, p_2, \dots, p_Z$! We can find these roots by testing whether $P(x) == 0$ for all x from 1 to N .

Subtask 2: $Z \leq 100$, $L = 0$, $Q = 200$

When we know **where** the islands are, finding **how much** gold they have is relatively easy. We plug our known islands p_k back into our first equation:

$$a_1(p_1)^1 + a_2(p_2)^1 + \dots + a_{Z(p_Z)}^1 = S_1$$

$$a_1(p_1)^2 + a_2(p_2)^2 + \dots + a_{Z(p_Z)}^2 = S_2$$

...

This is a system of linear equations (called the vandermonde matrix), which we can solve for $a_1 \dots a_Z$ using gaussian elimination.

Subtask 3: $Z \leq 5$, $L \leq 15$, $Q = 400$

The scallywag Pirate now lies...

But, Z is very small, and we have a lot of queries.

We can bypass the lies entirely using majority voting.

If we want to safely know the answer for a query X , we can ask it $2L + 1 = 31$ times, and let the answer to the query be the most common response.

Since the Pirate can only lie 15 times, the strict majority of our 31 candidate answers is guaranteed to be true!

We need $2Z = 10$ power sums, requiring $10 \times 31 = 310$ queries. We then feed these clean power sums to the algorithm from Subtask 2.

Subtask 4: $N \leq 1000$, $Z \leq 40$, $L \leq 40$, $Q = 160$

Voting now requires too many queries.

The query limit $Q = 2Z + 2L$ is exactly the information theoretical bound, it is impossible to do it better.

Now we are forced to use error correction codes.

We can construct our queries so that if the Pirate doesn't lie, the answers form the evaluations of a polynomial of degree $< 2Z$.

With up to L lies, this is pretty much exactly Reed solomon error correction.

We can use Welch Berlekamp. This sets up a linear system of size Q to find an error locator polynomial $E(x)$ and an evaluator polynomial $N(x)$.

By doing polynomial long division $\frac{N(x)}{E(x)}$, the lies are removed perfectly, leaving us with the real power sums.

Since $Q \leq 160$, solving this system with gaussian elimination in $O(Q^3)$ is speedy. And since $N \leq 1000$, finding the roots of the final Berlekamp-Massey polynomial can be done by simply testing $1, 2, \dots, N$.

Full solution: $N \leq 10^5$, $Z \leq 100$, $L \leq 100$, $Q = 400$

We combine everything.

1. $Q = 2Z + 2L$ queries structured as Reed solomon code, combined with Vandermonde moments
2. Solve the welch Berlekamp linear equations in $O(Q^3)$ to correct the Pirate's lies and extract the exact power sums
3. Run the Berlekamp massey algorithm in $O(Z^2)$ to construct the characteristic polynomial
4. Find the roots in $O(N \cdot Z)$ using Chien Search or fast polynomial evaluation to locate the islands
5. Solve the final $Z \times Z$ linear system to find the exact gold amounts

Cheese: the judges put a lot of work into making the pirate really adversarial. If you attempt to use probabilistic algorithms like sparse peeling, IBLTs, or sliding windows, it shouldn't pass. We are relatively confident that only the optimal $O(Q^3)$ decoding can survive all test cases.